

UNIDAD 3 - DOCKER: LA RECETA REPRODUCIBLE

Taller de aula: leer una receta que todavía no corriste

Programación 3 - Tecnicatura Universitaria en Programación - UTN FRM.

Modalidad	En grupos de dos o tres, en el aula. Se entrega antes de irse : lo que no se termina en la mesa no se termina.
Requisitos	No hace falta tener Docker instalado. Tampoco hace falta internet. Solo el apunte del capitulo 3 a mano.
Tiempo	70 minutos. Partes 1 a 5, mas la Parte 6 que se lleva al proyecto.
Puntaje	100 puntos: 1 (20) + 2 (15) + 3 (20) + 4 (15) + 5 (20) + 6 (10).
Que se evalua	Criterio de construccion: decidir que entra a la imagen, en que orden y por que, y poder justificar cada linea con el mecanismo que la explica.

Integrantes: Avila Lucas, Pagano Amanda, Liendo Mateo.

Parte 1 - Inventario de instrucciones: que pesa y que solo anota

20 puntos

Este es el archivo de construcción del backend del proyecto, sin los comentarios y con las líneas numeradas. Es la misma receta que Easypanel va a ejecutar en el servidor en la Clase 4.

```
# backend/Dockerfile
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY main.py .
6 COPY db.py .
7 EXPOSE 8000
8 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

1.1 Completa el inventario

12 puntos (2 por fila)

Una fila por instrucción. En "Que la invalida" escribí el cambio más chico que obliga a volver a ejecutarla. En la última columna, qué pasaría si borras esa línea del archivo.

Línea	¿Capa o metadato?	Que hace	Que la invalida (el cambio más chico)	Si la borro del archivo...
1	Capa que se hereda	Establece la imagen	Cambiar un byte del texto. La clave del caché acá es el texto literal de la instrucción.	No se puede construir. Toda imagen parte de otra.
2	metadato	Fija el directorio de trabajo	Cambiar el texto de la ruta. Es metadato, pero su texto igual es clave de caché.	Todo se ejecuta en / y el contenedor arranca y muere
3, 4	Capa	Copy agrega archivos al sistema. Run ejecuta un comando y guarda el resultado	un byte de requirements.txt · 4: su texto, o que se invalide la 3	3: falla la construcción · 4: construye, pero el contenedor arranca y muere
5, 6	Capa	Agrega archivos al sistema de archivos	Un byte de main.py / db.py	Construye, pero el contenedor arranca y muere
7	Metadato	Documenta un puerto	Cambiar el número	No pasa nada: solo se pierde documentación
8	Metadato	Declara el proceso de arranque	Cualquier carácter del texto	Construye, pero no hay proceso: hereda el CMD de la imagen base

1.2 Predecir la salida de docker history

4 puntos

Sin correr nada: de las ocho líneas, cuántas van a aparecer con un tamaño distinto de 0 B?

Nombralas y ordénalas de mayor a menor peso esperado, con una línea de justificación para la primera.

Van a aparecer cinco líneas con tamaño distinto a 0MB:

1. Línea 1– FROM Python 3.12-slim-> Descarga e incorpora un sistema operativo Debian completo de Python con sus librerías estándar, tiene un peso de 150MB aprox
2. Línea 4– RUN pip install-no-cache-dir-r requirments.txt
3. Linea 5– COPY main.py

4. Línea 6– COPY db.py
5. Línea 3– COPY requirements.txt

1.3 La instrucción que no está

4 puntos

El archivo real del proyecto copia main.py y db.py por separado, en dos líneas, pudiendo escribir COPY . . en una sola. Da las dos razones por las que se eligió la versión larga. Una es de rendimiento y la otra es de otra categoría: decime cual es cual.

Razón de rendimiento (caché): con COPY . . la clave de la capa es el resumen de todo el contexto de construcción. Cualquier archivo que cambie, un README, un log, un test, un .pyc invalida esa capa y, por la propiedad en cascada, todas las siguientes. Nombrando los archivos, solo se invalida cuando cambia el código que realmente entra en la imagen.

Razón de otra categoría seguridad (filtración de secretos): COPY . . mete en la imagen todo lo que esté en el directorio: .env, .git/, credenciales, claves. Y por el modelo de capas de la sección 3.3.2, una capa no se puede editar, solo tapar: si después borrás el .env en una instrucción posterior, la capa que lo contiene sigue en la imagen y cualquiera puede extraerlo. Copiar por nombre es una lista blanca explícita: solo entra lo que nombraste.

Parte 2 - La cascada del cache

15 puntos

Un grupo construyo la imagen del backend y despues hizo tres cambios, uno por vez, reconstruyendo despues de cada uno. Estas son las tres salidas, recortadas a lo que importa. Nadie toco requirements.txt salvo donde se indica.

```
--- Construccion A: se cambio una linea de codigo en main.py ---
=> [1/6] FROM python:3.12-slim                                CACHED
=> [2/6] WORKDIR /app                                          CACHED
=> [3/6] COPY requirements.txt .                                CACHED
=> [4/6] RUN pip install --no-cache-dir -r requirements        CACHED
=> [5/6] COPY main.py .                                       0.1s
=> [6/6] COPY db.py .                                         0.1s
tiempo total: 1.9s

--- Construccion B: se agrego una dependencia a requirements.txt ---
tiempo total: 118.4s

--- Construccion C: se renombro WORKDIR /app a WORKDIR /aplicacion ---
tiempo total: 121.7s
```

2.1 Traza las tres construcciones

8 puntos

Complete la tabla marcando CACHED o EJECUTA en cada celda, y marca con un círculo, en las columnas B y C, la instrucción exacta donde se rompió la cadena.

#	Instrucción	A: cambio main.py	B: cambio requirements.txt	C: renombro WORKDIR
1	FROM python:3.12-slim	CACHED	CACHED	CACHED
2	WORKDIR	CACHED	CACHED	EJECUTA (se rompe la cadena
3	COPY requirements.txt .	CACHED	EJECUTA (se rompe la cadena	EJECUTA
4	RUN pip install ...	CACHED	EJECUTA	EJECUTA
5	COPY main.py .	EJECUTA	EJECUTA	EJECUTA
6	COPY db.py	EJECUTA	EJECUTA	EJECUTA

2.2 Explicalo en una frase de ingeniería

4 puntos

Completa: "Renombrar el directorio de trabajo volvió a descargar todas las dependencias porque la clave del caché de una instrucción es el texto literal de la instrucción más la identidad , y porque cuando una capa se invalida todas las siguientes se invalidan también."

2.3 El comando que muestra la verdad de adentro

3 puntos

Nombra el comando que permite leer, capa por capa, que instruccion creo cada una y cuanto pesa. Y explica que cosa muestra ese comando que ninguna lectura del Dockerfile puede mostrarte.

El comando es docker history.
Muestra el peso real de cada capa y lo que quedó efectivamente dentro de la imagen, incluidas las capas heredadas de la imagen base. El Dockerfile es la intención; docker history es el resultado. Por eso ahí se ve que un archivo borrado en un paso posterior sigue presente: una capa no se puede editar, solo tapar

Parte 3 - El orden de la receta

20 puntos

Doce pasos, desordenados, para construir la imagen de una API en Python que necesita una biblioteca del sistema y que no debe correr como administrador. Once son instrucciones del `Dockerfile`; **uno no lo es**, y donde va ese es la mitad del ejercicio. El orden aca no es cuestion de estilo: hay posiciones que rompen la construccion, posiciones que rompen el despliegue de todos los días y una que quema una credencial para siempre.

```
a) COPY main.py .
b) RUN pip install --no-cache-dir -r requirements.txt
c) FROM python:3.12-slim
d) USER aplicacion
e) COPY requirements.txt .
f) WORKDIR /app
g) CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
h) RUN apt-get update && apt-get install -y --no-install-recommends \
    libpq5 && rm -rf /var/lib/apt/lists/*
i) EXPOSE 8000
j) RUN useradd --create-home aplicacion
k) escribir el archivo .dockerignore con .git/, .venv/, __pycache__/ y .env
l) ENV PYTHONUNBUFFERED=1
```

3.1 Ordena los doce pasos

10 puntos

Escribí las doce letras en el orden correcto de ejecución.

- 1- K)escribir el archivo .dockerignore con .git/, .venv/, __pycache__/ y .env
- 2-C)FROM Python:3.12-slim
- 3-I)ENV PYTHONUNBUFFERED=1
- 4-H)RUN apt-get update && apt-get install --no-install-recommends \ libpq5 && rm -rf /var/lib/apt/lists/*
- 5- J)RUN useradd--create-home aplicacion
- 6- F)WORKDIR /app
- 7- E)COPY requirements.txt .
- 8- B)RUN pip install-- nocache-dir-r requirements
- 9-A)COPY main.py .
- 10-D)USER aplicacion
- 11-i)EXPOSE 8000
- 12-G) CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Tabla de justificación

- K es la primera porque debe existir antes de cualquier comando COPY o Docker build, sino un pequeño descuido podría mandar el .env con secretos del contexto de construcción
- H y J La instalación del sistema (libpq5) y la creación del usuario del sistema deben ejecutarse antes de cambiar al usuario no privilegiado
- E y B Copian primero requirements.txt y ejecuta pip install para aprovechar la cache de Docker

- D Las dependencias se instala a nivel de sistema/root o con los permisos necesarios sin tener problemas de escritura
- I y G definen como se expone y ejecuta el contenedor en runtime

3.2 Marca los tres puntos de no retorno

6 puntos (2 por punto)

Tres de esos pasos, puestos fuera de lugar, hacen daño de tres clases distintas: uno rompe la construcción en el acto, uno no rompe nada y te cuesta minutos en cada despliegue, y uno no se puede deshacer reescribiendo el archivo. Identificarlos y decime, en cada caso, qué error concreto los convierte en trampa.

Paso	Clase de daño	Que error lo convierte en trampa
d) USER aplicacion	Rompe la construcción en el acto	Si se adelanta antes de j) RUN useradd o de los RUN que instalan el build falla al instante: antes de j) el usuario no existe todavía, y h) apt-get y b) pip install necesitan privilegios (Permission denied). El apunte lo dice literal en 3.9.3: USER debe ir después de las instrucciones que necesitan privilegios instalar paquetes, cambiar permisos y antes del CMD.
a)COPY main.py	No rompe nada y te cuesta minutos en cada despliegue	Si sube por encima de e) y b), la imagen funciona igual, pero cada cambio en el código invalida esa capa y, en cascada, el pip install: se vuelven a descargar todas las dependencias en cada despliegue
k) escribir el .dockerignore	No se puede deshacer reescribiendo el archivo	Si no existe antes de construir, el .env entra en el contexto y queda en una capa. Borrarlo después no lo elimina, lo tapa: hay que rotar la credencial

3.3 Los que no pesan y aun asi importan

4 puntos

Tres de los once pasos que si son instrucciones no crean ninguna capa. Identifica y explica por qué su posición en el archivo igual importa, si no pesan nada.

Los tres pasos son

- f) WORKDIR /app,
- l) ENV PYTHONUNBUFFERED=1 y
- d) USER aplicacion

son instrucciones de metadato, no modifican el sistema de archivos y por eso no crean capa.

Su posición importa porque no ejecuta nada, pero cambian el contexto en el que corren todas las instrucciones que vienen abajo: WORKDIR fija el directorio de las siguientes, ENV define la variable que ven las siguientes, y USER fija con qué privilegios se ejecutan las siguientes. Además su texto es clave de caché: cambiarlo invalida todo lo que sigue. Pesan cero, pero su alcance es todo lo que está debajo.

Parte 4 - Auditoria de un archivo de construccion

15 puntos

Un grupo entrego este Dockerfile como "listo para desplegar". Tiene **cinco problemas reales** -uno de ellos con dos defectos distintos en la misma linea- y **una linea que parece un problema y no lo es**.

```
# Dockerfile entregado por un grupo
1 FROM python:latest
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 RUN apt-get update
6 RUN apt-get install -y curl vim git
7 ENV DATABASE_URL=postgres://postgres:tup2026@db:5432/tup
8 EXPOSE 8000
9 CMD uvicorn main:app --host 127.0.0.1 --port 8000
```

4.1 Los cinco problemas

10 puntos (2 por problema)

Para cada uno: número de línea, corrección concreta y una línea de justificación. Sin justificación no suma.

Línea	Corrección	Justificación
1	Cambiar la versión de Python a una mas ligera (por ejemplo Python:3.13-slim)	Cuando se utiliza el "latest" estamos siempre descargando una imagen de la ultima versión de Python la cual pesa aprox 1GB arruinando la reproducibilidad del build
3	Reemplazar por COPY requirements.txt	Copiar todo el directorio invalida la cache del Docker, forzando la reinstalación de las librerías en cada build
5 y 6	Borrar paquetes innecesarios (vim, git) y unificar el comando en una sola instrucción	Si se separa el update y install en dos RUN, la cache de update se congela, además que si instalasen herramientas interactivas de depuración en producción engordan la imagen y amplia la superficie para atacantes
7	Borrar ENV DATA_BASE URL= . . .	Hardcodear credenciales dentro de la instrucción del Dockerfile se quedan grabadas de forma permanente e insegura en el historial de capas de la imagen
9	Cambiar el host por 0.0.0.0	El host 127.0.0.1 limita la escucha a la interfaz interna del contenedor, haciendo que la API sea completamente inaccesible des el host o exterior
Línea que parece un problema pero no lo es	La línea 8 EXPOSE 8000	Es una directiva de información/documentación que no publica ni bloquea puertos

4.2 La línea que no es un problema

5 puntos

Un integrante lee en el apunte que EXPOSE no abre ningún puerto y propone borrar la línea 8, "porque no hace nada". Decime si tiene razón en el hecho y por que igual la decisión es incorrecta. Nombra además quien consume esa línea.

Tiene razon en el hecho por que el EXPOSE no abre nada, pero de todas formas es una decisión incorrecta porque si se borra el contenedor, aunque funcione exactamente, igual vas a confundir a la próxima persona que lo intente leer y quien consume esa línea es Easypanel

Parte 5 - Peritaje: el contenedor que arranca y no anda

20 puntos

Un grupo pide ayuda: "construimos las dos imagenes, los contenedores estan arriba, el log no tira ni un error y la calculadora no calcula". Un rato despues escriben de nuevo: "ah, y nos avisaron que la contrasena de nuestra base aparece publicada". Esta es toda la evidencia que trajeron. Leela entera antes de escribir nada.

```
$ docker ps
CONTAINER ID   IMAGE     STATUS   PORTS                               NAMES
a41c9f2b7e30   tup-api   Up 12 minutes   0.0.0.0:8000->8000/tcp   tup-api
7d2e5a1c9004   tup-web   Up 12 minutes   0.0.0.0:8080->80/tcp     tup-web

$ docker logs tup-api | tail -3
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)

$ curl http://localhost:8000/api/salud
curl: (52) Empty reply from server

$ docker logs tup-web | tail -2
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/08/28 03:02:11 [notice] 1#1: start worker processes

$ docker exec tup-api ls -la /app
.  ..  main.py  db.py  requirements.txt

$ docker history tup-api
HISTORIC
CREATED BY                                     SIZE
CMD ["sh" "-c" "uvicorn main:app --host 127.0..."]   0B
RUN /bin/sh -c rm -f .env                         0B
COPY . . # buildkit                               48.2MB
RUN /bin/sh -c pip install -r requirements.txt       61.4MB
WORKDIR /app                                       0B
FROM docker.io/library/python:latest              997MB

$ docker save tup-api -o api.tar && tar -xf api.tar
$ grep -r "DATABASE_URL" blobs/
blobs/sha256/9f3cle...: DATABASE_URL=postgres://postgres:tup2026@db:5432/tup

$ time docker stop tup-api
tup-api
real    0m10.412s
```

5.1 Reconstruir la cadena de hechos

6 puntos

En seis líneas como máximo, contame que paso, en orden, desde que el grupo escribió el Dockerfile hasta que la contraseña terminó publicada. Sostene cada afirmación en una línea concreta de la evidencia.

1. Escribieron el Dockerfile con COPY . . y sin .dockerignore history: COPY . . # buildkit 48.2MB.
2. En el contexto de construcción había un .env con la credencial, que entró en esa capa junto con el código docker save + grep: DATABASE_URL=postgres://postgres:tup2026@db:5432/tup.
3. Después intentaron taparlo borrándolo history: RUN /bin/sh -c rm -f .env 0B.

4. Ese borrado creó una capa nueva que solo marca el archivo como ausente; por eso `docker exec tup-api ls -a /app` no muestra ningún `.env`.
5. La capa que lo contiene sigue dentro de la imagen y es extraíble con herramientas comunes el `grep` sobre `blobs/sha256/9f3c1e...` lo encuentra.
6. La imagen viajó con el secreto adentro: la contraseña quedó publicada.

5.2 Las tres contradicciones aparentes

6 puntos (2 cada una)

La evidencia parece contradecirse tres veces. Explica cada caso: (i) el log dice Uvicorn running y no hay un solo error, pero `curl` no obtiene respuesta; (ii) `docker exec ls -a` no muestra ningún `.env` dentro del contenedor, y sin embargo la credencial se extrae de la imagen; (iii) el contenedor del frontend arrancó sin errores y la calculadora igual no funciona.

- (i) El log dice Uvicorn running y `curl` no obtiene respuesta. No hay error porque no hubo fallo: el proceso arrancó bien. Escucha en `127.0.0.1`, que es el bucle local del contenedor tiene su propio espacio de nombres de red, un lugar al que nadie de afuera puede llegar, ni siquiera el anfitrión. Por eso `curl` recibe Empty reply en vez de un error.
- (ii) `ls -a` no muestra el `.env` pero la credencial se extrae de la imagen. El contenedor ve el sistema de archivos de unión resultante, donde la capa del `rm` tapa al archivo. La capa que lo contiene no se borró: sigue en la imagen. Una capa no se puede editar, solo tapar.
- (iii) El frontend arrancó sin errores y la calculadora no funciona. El frontend no ejecuta nada en el servidor: solo entrega archivos, y el JavaScript corre en el navegador del visitante. `nginx` arrancó bien porque hizo su trabajo; el que no responde es el backend.

5.3 Los principios violados

4 puntos

Nombra los principios del capítulo que el grupo violó y en qué decisión concreta. Uno de ellos ya lo viste en la Clase 2 y aparece acá aplicado dentro de la imagen: decime cual y como se traduce.

El grupo violó los siguientes principios fundamentales de ingeniería de contenedores:

1. **Principio de inmutabilidad de capas (una capa no se puede editar, solo tapar):**
Decisión concreta: Intentar borrar el archivo confidencial con `RUN /bin/sh -c rm -f .env` en una capa posterior. Justificación: En el modelo de almacenamiento UnionFS, cada capa es inmutable y de solo lectura. Un `rm` posterior solo agrega un registro de supresión (whiteout) en la capa superior, pero no borra los datos de la capa original, permitiendo que cualquiera extraiga los secretos inspeccionando los blobs.
2. **Principio de separación de código y configuración / Gestión de secretos en runtime:**
Decisión concreta: Incluir el archivo `.env` en el contexto de construcción al usar `COPY` . . sin un archivo `.dockerignore`. Justificación: La configuración variable y las credenciales deben desacoplarse del ciclo de vida de la imagen e inyectarse en memoria durante el tiempo de ejecución (runtime) mediante variables de entorno en el orquestador (Easypanel), nunca quedar grabadas en el artefacto distribuible.

3. Principio de aislamiento de red y binding de interfaces (visto en la Clase 2):

Decisión concreta: Configurar el servidor de la API con --host 127.0.0.1. Cómo se traduce adentro de la imagen: En la Clase 2 se vio que los contenedores tienen su propio espacio de nombres de red (Network Namespace) aislado del anfitrión.

Adentro de la imagen, este principio se traduce en que todo proceso servidor que deba atender solicitudes externas tiene la obligación técnica de escuchar en 0.0.0.0 (todas las interfaces de red del contenedor). Al escuchar en 127.0.0.1, el servidor atiende exclusivamente al loopback interno del contenedor y rechaza cualquier paquete reenviado por el host a través del mapeo de puertos (-p 8000:8000).

4. Principio de proceso principal como PID 1 y manejo de señales POSIX:

Decisión concreta: Utilizar la sintaxis en formato shell para el arranque: CMD uvicorn main:app ... (que en docker history figura como CMD ["sh", "-c", ...]). Justificación: En formato shell, el proceso que asume PID 1 es /bin/sh, el cual no transmite las señales del sistema operativo (SIGTERM) a los procesos hijos. Por esta razón, en la evidencia time docker stop tup-api demoró exactamente 10.412 segundos: Docker esperó el timeout de 10 segundos antes de verse obligado a enviar un SIGKILL no controlado. Debe usarse siempre la sintaxis exec: CMD ["uvicorn", "main:app", ...].

5. Principio de reproducibilidad y superficie mínima: Decisión concreta: Utilizar FROM python:latest. Justificación: latest es un puntero mutable que destruye la reproducibilidad del build e introduce paquetes de compilación innecesarios (ampliando el peso a casi 1 GB y agrandando la superficie de ataque).**5.4 Las cuatro correcciones, en orden****4 puntos**

Escribí las cuatro medidas que aplicarías, en el orden en que las aplicarías. Justifica el orden, no solo el contenido: una de las cuatro no puede esperar a las otras tres.

Las cuatro medidas en estricto orden de aplicación son:

- 1. Medida 1 (Inmediata / Emergencia de seguridad):** Rotar y revocar inmediatamente la contraseña de la base de datos en el servidor y generar una credencial nueva y segura. Justificación del orden: Esta medida no puede esperar a las otras tres bajo ninguna circunstancia. La credencial postgres:tup2026@db:5432/tup ya fue publicada en un archivo tar y en las capas de la imagen. Aunque se arregle el Dockerfile o se recompile la imagen, si la contraseña original sigue activa en la base de datos el sistema sigue comprometido. Neutralizar la vulnerabilidad en el origen es la prioridad número cero.
- 2. Medida 2 (Aislamiento del contexto):** Crear y configurar el archivo .dockerignore en la raíz del proyecto, excluyendo explícitamente .env, .env.*, .git/, .venv/, __pycache__/ y tests/. Justificación del orden: Debe realizarse antes de cualquier nueva construcción de imagen. Si no se aísla el contexto primero, cualquier nuevo intento de docker build volvería a absorber los archivos confidenciales del entorno local y los grabaría en una nueva capa.

3. **Medida 3 (Corrección de la receta técnica):** Refactorizar el Dockerfile corrigiendo los defectos técnicos: (a) Fijar la imagen base ligera y reproducible FROM python:3.12-slim; (b) Copiar dependencias primero (COPY requirements.txt .) y ejecutar RUN pip install --no-cache-dir para aprovechar la caché; (c) Copiar solo el código necesario (COPY main.py . y COPY db.py .); (d) Eliminar definitivamente RUN rm -f .env y directivas ENV con claves hardcodeadas; (e) Cambiar el comando de arranque a formato exec escuchando en todas las interfaces: CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]. Justificación del orden: Con el contexto seguro y las credenciales viejas invalidadas, se construye la receta técnica adecuada.
4. **Medida 4 (Reconstrucción, inyección en runtime y validación):** Reconstruir la imagen desde cero (docker build -t tup-api .), desplegar el contenedor inyectando la nueva contraseña por variables de entorno en tiempo de ejecución (a través de Easypanel o con -e DATABASE_URL=...), y verificar la conectividad con curl http://localhost:8000/api/salud y el apagado limpio con docker stop. Justificación del orden: Es el paso final de verificación y puesta en producción una vez que todas las etapas anteriores garantizaron la seguridad y la corrección técnica.

Parte 6 - El plan de imagenes de tu propio proyecto

10 puntos

Esta parte no es un ejercicio de laboratorio: es el plano de lo que Easypanel va a construir en las Clases 4 y 5. Retoma la tabla de puertos de la Parte 6 del taller anterior, un nivel más adentro: allá decidiste que se publica; acá decidís que entra a cada imagen, en que orden y que se queda afuera.

Servicio	Imagen base con etiqueta exacta	Que se copia y en qué orden	Que NO entra (.dockerignore)	Que decide el entorno	Puerto interno
Frontend	nginx:1.27-alpine	1. COPY nginx.conf /etc/nginx/conf.d/de fault.conf 2. COPY css/ /usr/share/nginx/html/css/ 3. COPY index.html app.js /usr/share/nginx/html/	.git/ .gitignore node_modules/ README.md .env* Dockerfile tests/	URL base de la API del backend (o configuración de proxy reverso /api en Nginx hacia la API interna).	80
API	python:3.12-slim	1. COPY requirements.txt . 2. RUN pip install --no-cache-dir -r requirements.txt 3. COPY db.py . 4. COPY main.py .	.env .env.* .git/ .venv/ __pycache__ / *.pyc *.pyo .pytest_cache / tests/ README.md Dockerfile .dockerignore	DATABASE_URL (credenciales y host de Postgres: postgresql://user:password@db:5432/tup) PORT LOG_LEVEL Claves de sesión / JWT	8000
Base de datos	postgres:16-alpine	No requiere Dockerfile propio en Easypanel (se usa la imagen oficial). Si requiere inicialización: 1. COPY init.sql /docker-entrypoint initdb.d/	Backups (*.dump, *.sql) .env .git/ Archivos de datos locales (pgdata/)	POSTGRES_DB (ej. tup) POSTGRES_USER (ej. postgres) POSTGRES_PASSWORD (credencial generada en Easypanel) PGDATA	5432

6.1 La decisión que arrastra a todas las demás

5 puntos

De todo lo que escribiste en tu tabla, una sola celda por fila condiciona a las otras cinco: si la cambias mas adelante, tenes que revisar la fila entera. Nombrala, decime por que, y explica qué problema concreto te evita fijarla hoy con su etiqueta exacta.

- Nombre de la celda: La celda de Imagen base con su etiqueta exacta (por ejemplo python:3.12-slim, nginx:1.27-alpine o postgres:16-alpine).
- Por qué condiciona a las otras cinco: La imagen base define todo el entorno operativo y el contrato sobre el que se apoya el resto de las decisiones: determina la distribución Linux subyacente (Debian vs. Alpine), la implementación de la biblioteca C del sistema (glibc en Debian vs. musl en Alpine, lo cual condiciona si paquetes de Python como psycopg2 o asyncpg se instalan como wheels precompilados o requieren herramientas de compilación C que multiplican el tiempo de build), el gestor de paquetes del sistema (apt vs. apk), las rutas predeterminadas de trabajo (/app, /usr/share/nginx/html), el usuario predeterminado de ejecución y el puerto interno que expone nativamente (80, 8000, 5432). Si se cambia la imagen base más adelante, deben reescribirse las instrucciones de instalación, los destinos de copia, las dependencias y la configuración de puertos.
- Qué problema concreto te evita fijarla hoy con su etiqueta exacta: Evita el problema de las roturas silenciosas por actualización flotante (version drift) y garantiza el determinismo absoluto del build. Si se usa una etiqueta flotante como latest o simplemente python:3, el proveedor de la imagen puede actualizar la versión del intérprete (por ejemplo de 3.12 a 3.13) o las librerías base en cualquier momento. Eso produce que un despliegue que funcionaba perfectamente ayer falle de repente hoy en Easypanel sin que el grupo haya modificado una sola línea de código fuente. Fijar la etiqueta exacta garantiza que la construcción sea 100% reproducible tanto en local como en el servidor de producción.

6.2 La línea que te vas a agradecer

5 puntos

Mira tu tabla y responde: si mañana alguien de tu grupo dice "metamos el env dentro de la imagen así no hay que configurar nada en Easypanel y desplegamos más rápido", ¿que le decís? Escribe la respuesta técnica y la alternativa concreta que le ofreces, porque la necesidad es legítima.

Respuesta técnica: Le digo que no rotundo, porque esa decisión rompe dos principios fundamentales de ingeniería de software:

1. Seguridad e inmutabilidad: Una imagen de Docker es un paquete compilado distribuible. Si metemos el .env, las credenciales quedan impresas de manera permanente e imborrable en el historial de capas de la imagen. Como quedó probado en el peritaje de la Parte 5, un borrado posterior con rm no elimina el archivo (solo lo tapa con un whiteout), y cualquiera con acceso al registro o descargando la imagen con docker save puede extraer las contraseñas en texto plano con un simple grep.
2. Principio de configuración desacoplada (Twelve-Factor App): La imagen debe ser un artefacto universal. Si contiene el .env, queda rígidamente atada a un único entorno (la base local de desarrollo). No se puede promover esa misma imagen a staging o producción, y ante cualquier necesidad de rotar una credencial estaríamos obligados

a recompilar y redistribuir la imagen completa, en vez de cambiar una variable en el servidor.

Alternativa concreta: Entendiendo que su necesidad legítima es la agilidad y evitar la carga manual repetitiva de campos en la interfaz de Easypanel, la alternativa profesional es:

1. Versionar en el repositorio un archivo `.env.example` que documente todas las variables necesarias con valores de ejemplo o marcadores de posición (sin secretos reales).
2. En Easypanel, utilizar la función de carga masiva de variables de entorno (Bulk Edit / Raw Input) en la solapa "Environment", pegando el bloque de configuración real una única vez. O mejor aún, aprovechar las referencias dinámicas de Easypanel vinculando directamente la variable provista por el contenedor de base de datos (`{db.DATABASE_URL}`). De este modo, las contraseñas se inyectan en memoria exclusivamente en tiempo de ejecución (runtime), el despliegue es igual de rápido, y la imagen permanece limpia, segura y 100% portable.

Cierre - Para discutir en la mesa

Sin puntaje

La conclusión del capítulo es una sola frase: la receta es la misma en tu máquina y en el servidor. No parecida, no equivalente, no "casi igual salvo un par de cosas". La misma. Desde el momento en que el entorno se escribe en un archivo versionado, "en mi maquina andaba" deja de ser una explicación: la maquina paso a ser parte del código, y por lo tanto es revisable, comparable y discutible como cualquier otra línea que hayan escrito.

Síntesis de discusión del grupo: En los proyectos anteriores de la carrera, gran parte del entorno vivía de manera informal en la memoria de algún compañero o en notas personales: la versión exacta del intérprete instalada a mano, las librerías del sistema agregadas sin registrar, las credenciales compartidas por chat privado y las tablas creadas manualmente en una consola. Cada uno de esos elementos indocumentados representaba un punto único de falla que detonaba en el servidor. Al adoptar Docker, toda esa infraestructura invisible se convierte en código auditable en el repositorio: el runtime es FROM python:3.12-slim, las dependencias de sistema son el RUN apt-get, los módulos de Python residen en requirements.txt y la configuración dinámica se inyecta en variables de entorno en runtime. Si no está en la receta, no existe; y si está en la receta, funciona exactamente igual en cualquier entorno.